

Developer Tools Execution Report

Interactive JavaScript Console Verification (Tasks 4, 5, & 6)

Target Environment: Google Chrome DevTools | Platforms: Instagram, Zomato, Flipkart

Executive Summary

This professional visualization report documents the empirical verification of client-side JavaScript execution and browser runtime behavior across three major web platforms: **Instagram**, **Zomato**, and **Flipkart**. Utilizing Google Chrome Developer Tools, diagnostic commands were injected into active DOM environments. The captured telemetry highlights successful runtime evaluation alongside crucial network security policies and client-side request interception.

Task	Platform	Input Expression	Console Output	Primary Diagnostic
Task 4	Instagram	console.log("Hello from the console")	Hello from the console	Tracker POST blocked
Task 5	Zomato	console.log("Hello from the console!")	Hello from the console!	Permissions policy enforcement
Task 6	Flipkart	console.log("JS is running!")	JS is running!	Unused preload script detected

Table 1: Executive Overview of Console Verification Tasks

1 Task 4: Instagram Console Verification

Objective: Access Chrome DevTools on Instagram, navigate to the Console tab, and evaluate a runtime diagnostic log statement.

- **Target URL:** `https://www.instagram.com`
- **Executed Command:** `console.log("Hello from the console");`
- **Observed Behavior:** The browser successfully evaluated the expression, returning the expected string output followed by the default `undefined` return value.
- **Network Diagnostics:** Multiple asynchronous tracking and analytics POST requests targeting `/ajax/bz?` were intercepted and blocked by client-side privacy extensions (`net::ERR_BLOCKED_BY_CLIENT`).

Instagram DevTools Console Telemetry

```
[X] POST https://www.instagram.com/ajax/bz?__a=1&__cgc=GOOD... ONgMS-WPRTe.js:288
net::ERR_BLOCKED_BY_CLIENT [X] POST https://www.instagram.com/ajax/bz?__a=1&__cgc=GOOD...
ONgMS-WPRTe.js:288 net::ERR_BLOCKED_BY_CLIENT > console.log("Hello from the
console") Hello from the console VM505:1 < undefined
```

2 Task 5: Zomato Console Verification

Objective: Open Chrome DevTools on Zomato, access the Console tab, and execute a custom greeting message.

- **Target URL:** `https://www.zomato.com`
- **Executed Command:** `console.log("Hello from the console!");`
- **Observed Behavior:** Immediate script execution yielding the exact string payload in the active virtual machine context (`VM357:1`).
- **Network Diagnostics:** A Permissions Policy violation warning was logged regarding restricted `unload` event handlers, alongside blocked event telemetry requests to `jumbo.zomato.com` and Google logging endpoints.

Zomato DevTools Console Telemetry

```
[X] [Violation] Permissions policy violation: unload is not allowed zomato-
5b4c68b.js:1 [X] POST https://jumbo.zomato.com/event zomato-5b4c68b.js:1
net::ERR_BLOCKED_BY_CLIENT > console.log("Hello from the console!") Hello from
the console! VM357:1 < undefined
```

3 Task 6: Flipkart JavaScript Execution

Objective: Open Chrome DevTools on Flipkart, access the Console tab, and verify execution integrity using a standard test script.

- **Target URL:** `https://www.flipkart.com`
- **Executed Command:** `console.log("JS is running!");`
- **Observed Behavior:** Successful console output verification. Concurrently, a resource preloading performance warning was triggered by the DOM engine for an unused image asset.
- **Network Diagnostics:** Business data collector endpoint requests (`sonic.fdp.api.flipkart.com`) were successfully blocked by client security layers.

Flipkart DevTools Console Telemetry

```
[X] POST https://sonic.fdp.api.flipkart.com/4/data/collector/business (index):2
net::ERR_BLOCKED_BY_CLIENT > console.log("JS is running!") JS is running!
(index):2 < undefined [!] The resource https://rukminim2.flixcart.com/... was
preloaded using link preload but not used within a few seconds. (index):1
```

4 Conclusion

The successful execution and logging of JavaScript expressions across Instagram, Zomato, and Flipkart confirm that client-side script evaluation is fully functional within the browser's developer environment. Furthermore, the captured console warnings and blocked telemetry requests provide valuable empirical data regarding modern web security policies, tracking prevention, and resource optimization.

Report compiled successfully via Automated LaTeX Environment